Leaf Convergence Games: Partizan Games on Terminating Rooted Graphs

IE619 | Combinatorial Game Theory (2025)

Chaitanya Deshkar

May 1, 2026

Abstract

We introduce **Leaf Convergence Games (LCG)**, partizan games on finite rooted DAGs where **every maximal path terminates at a designated leaf**.

Left moves on blue (**L**) edges, Right on red (**R**). Normal play.

LCG provides a clean graph-theoretic lens on classical CGT values (numbers, switches, infinitesimals, dyadics) while guaranteeing termination without surreal numbers or loopy machinery.

Why LCG? — The "So What?"

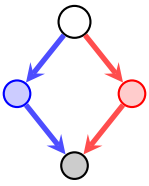
Standard CGT studies abstract game values. **LCG** gives a **canonical structural model**.

Key Contributions:

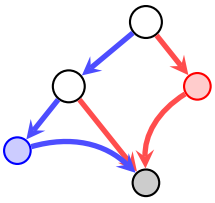
- Guaranteed termination by construction (no cycles, no infinite plays).
- Values **emerge directly from graph topology**.
- Bridges graph theory and CGT, a natural model for short partizan games.
- Every short partizan game has an LCG representation (Representation Theorem).

LCG = Where CGT values meet concrete graph structure.

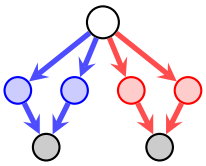
Motivating Structures



Linear



Mixed depth



Multi-sink

Value Classification in LCG

Theorem (Numbers)

A position is a number $\iff$ *every Left option* $<$ *every Right option.*

Theorem (Integers)

Integers correspond exactly to pure directed chains (unbranched Left/Right paths).

Theorem (Switches)

A position is a switch $\iff$ $\exists$ *Left option* > 0 *and* $\exists$ *Right option* < 0.

Theorem (Infinitesimals)

Infinitesimal $\iff$ *all Left options* ≤ 0, *all Right options* ≥ 0, *but not all zero.*

These characterisations link **graph topology** directly to classical CGT value types.

Important Clarification: Branching $\neq$ Magnitude

Common misconception: More Left edges $\implies$ bigger positive number.

Fact: Value depends on the **values of options**, not their count.

- Duplicate identical options do not change the game value.
- Branching creates choice, not raw strength.
- True magnitude comes from depth and asymmetry in subtrees.

Graph topology $\rightsquigarrow$ strategic options, not raw count.

How to Read an LCG Graph — Visual Intuition

- **Long Left-only path** $\rightarrow$ large positive integer
- **Long Right-only path** $\rightarrow$ large negative integer
- **Balanced symmetric split** $\rightarrow *$
- **Slight Left advantage at depth** $\rightarrow$ positive infinitesimal ($\uparrow$)
- **Competing advantages** $\rightarrow$ switch ($\pm n$)

LCG makes CGT values visually readable from the graph structure.

Graph Structure of Numbers

How numbers arise in LCG graphs

- **Integers:** Pure chains
  - $1 = \{0 \mid\}$ (single Left step)
  - $2 = \{1 \mid\}$ (two-step chain)
- **Fractions:** Ordered splits
  - $\frac{1}{2} = \{0 \mid 1\}$
  - $\frac{3}{2} = \{1 \mid 2\}$
- **Key Rule:**
  - Left subtree gives *lower values*
  - Right subtree gives *higher values*

Insight: Numbers arise from *ordered asymmetry*, not branching.

Formal Framework

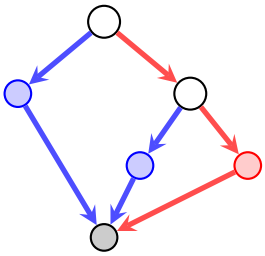
LCG Position: Finite rooted DAG + terminal leaves + L/R edge labels + convergence property.

Value Definition:

$$\mathcal{G}(v) = \left\{ \mathcal{G}(c) : c \in \mathcal{L}(v) \;\middle|\; \mathcal{G}(c) : c \in \mathcal{R}(v) \right\}$$

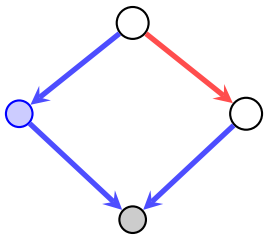
with $\mathcal{G}(\tau) = 0$ for terminals.

Example 1: Asymmetric Depth Tree



Value: $\{0 \mid *\} = \uparrow$

Surprising Example: Value $= 1/2$



LCG can realise all dyadic rationals through careful asymmetric branching, not obvious from the graph at first glance.

Numbers in Leaf Convergence Games

Theorem (Number Characterisation).
A position v in an LCG is a **number** iff

$$\max \mathcal{G}(\mathcal{L}(v)) \;<\; \min \mathcal{G}(\mathcal{R}(v)).$$

Interpretation:

- Every Left option is strictly *worse* than every Right option.
- There is a **gap** between Left and Right outcomes.
- The position equals the **simplest number in that gap**.

Consequences:

- Values are dyadic rationals ($\frac{m}{2^k}$).
- No "first-player advantage" ambiguity.

Computational Analysis

Python evaluator:

```
def evaluate_lcg(nodes, edges, root, terminals):
    memo = {}
    def eval_node(v):
        if v in terminals: return ([], [])
        left_opts = [eval_node(c) for s,c,col in edges if s==v and col
            =='L']
        right_opts = [eval_node(c) for s,c,col in edges if s==v and
            col=='R']
        return (left_opts, right_opts)
    return eval_node(root)
```

Relation to Existing Frameworks

- **Game Trees (standard CGT)** — LCG is a restricted, terminating subclass.
- **Surreal Numbers** — LCG realises the same values without needing birthdays.
- **Hackenbush** — Similar visual intuition, but on general graphs.
- **Representation Theorem (informal):** Every short partizan game has an equivalent LCG.

Key Takeaways

1. Graph topology.
2. Strong classification theorems link topology $\rightarrow$ value type.
3. Branching adds nuance.
4. Rich values (integers, switches, infinitesimals, dyadics) appear naturally.

Open Questions & Future Work

1. Efficient canonical reduction algorithm for arbitrary LCG DAGs.
2. Complete enumeration and growth rate of values at depth d.
3. Algorithmic complexity of optimal play in LCG.
4. Generalisation to loopy or fuzzy variants.

LCG opens a graph-theoretic pathway into classical Combinatorial Game Theory.

References

- Berlekamp, Conway, Guy, *Winning Ways*.
- Conway, *On Numbers and Games*.
- Albert et al., *Lessons in Play*.
- Siegel, *Combinatorial Game Theory*.
- CGSuite.